

Architectural styles

- An architectural style describes **how a system is organized**
- It defines:
 - What components exist
 - How they connect
 - How they communicate
 - How they form a complete system

What Is a Component?

- A component is a **part of a system with a defined interface**
- Examples: client, server, database, web service
- Components can be **changed without affecting the whole system**

What Is a Connector?

- A connector is a mechanism for **communication between components**
- Examples: HTTP request, RPC call, message queue, streaming connection

1- Layered architecture

- System is organized in **layers**, each layer has a specific role

a) Clean Structure

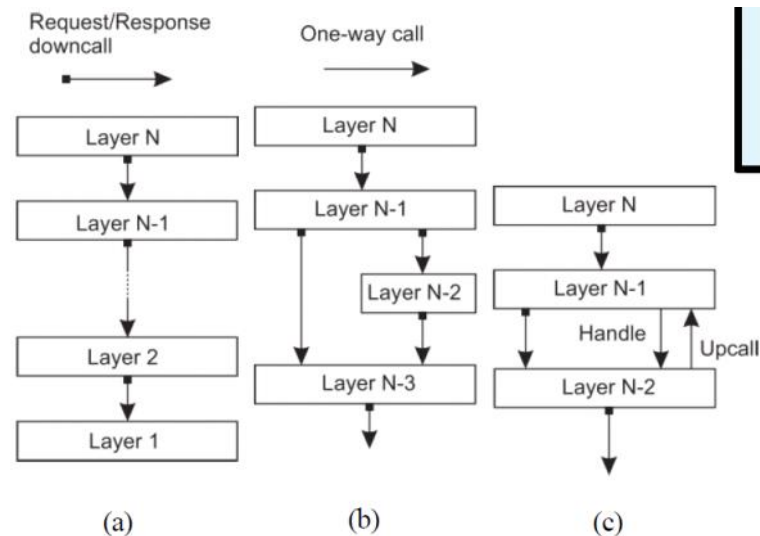
- Each layer communicates only with the **layer directly below**
- This structure is **clean and well-organized**

b) Flexible Sturture

- A layer can call **lower layers directly**, not only the next one
- This gives **flexibility** but reduces structure

c) Upcalls

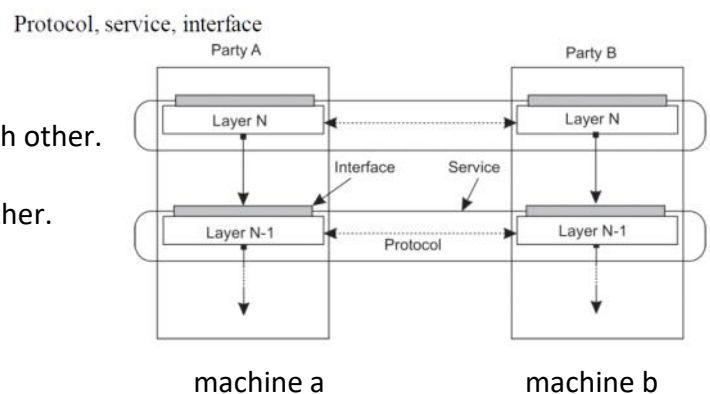
- A lower layer can call an **upper layer**
- Used to send information upward, **more complex**



Protocol: It is how layers on different machines talk to each other.

Interface: Method for adjacent layers interact with each other.

Service: Service is what a layer gives to the layer above it.



Application Layering

• Traditional systems use a **three-layer structure**, Traditional three-tiered view dividing systems into application-interface, processing, and data layers.

- Application-interface layer handles **user interaction**
- Processing layer handles **application logic**
- Data layer stores and manages **data**

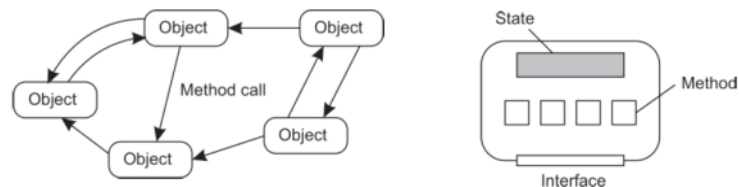
Example: Simple search engine uses UI, processing, and database layers

2- Object-Based Style

Object-Based System: A system constructed entirely from interacting objects.

Each object:

- Contains **data (state)**
- Provides **functions (methods)**
- Communicates using **method calls**



Object : An entity that contains data (state) and provides functions (methods).

Encapsulation: The concept where objects hide their internal implementation and only expose methods to manipulate their data. = data is **NOT seen** , methods is **SEEN**

- Objects communicate through **method calls** and
- hide internal data, which objects can be on the same machine or across different machines(remote calls).

3- REESTful Architecture

A style that views a distributed system as a collection of resources ,individually managed by components. Resources can be **created, read, updated, and deleted**.

Key Principles

- Everything Has a URL: Each resource has a **unique URL**
- Same Interface for All: All resources use the **same HTTP methods**
- Self-Described Messages: Each request contains **all necessary information** for the server
- **Stateless**: The system is **stateless**, each request is independent

Basic operations

- POST → create
- GET → read
- DELETE → remove
- PUT → update

Complexity in REST is shifted away from the simple interfaces and placed into **URI design, request parameters, and resource representations(XML/JSON)**.

Example: Amazon’s Simple Storage Service

Essence

- Objects (files) are stored in **buckets (directories)**
- Buckets cannot be inside other buckets
- Each object is accessed using a **URL**

Typical operations

- Create object → **PUT**
- List objects → **GET (on bucket name)**
- Read object → **GET (on full URL)**

! On interfaces

- REST interfaces are **simple** because they use standard operations (GET, PUT, POST, DELETE)
- Complexity is moved from the interface to **other parts of the system**

Where does the complexity move in REST?

- URI design
- Request parameters
- Resource representations (JSON/XML)
- Client-side logic and interpretation

4-Publish-subscribe architectures



Publish-Subscribe: A communication pattern where **publishers send events** and **subscribers** receive matching **events**.

Topic-Based Subscription: Subscribers match events by exact attributes and values. (“attribute = value”series) - basic-

Content-Based Subscription: Subscribers match events by an attribute range. (“attribute ∈ range”series) more flex

Coordination

- **Coordination styles** depend on **temporal coupling** and **referential coupling**

coupling = dependency
temporal = zaman
referential = birbirini tanıma

Temporal coupling means:

- Components must run at the **same time** → temporally coupled
- Components do NOT need to run at the same time → temporally decoupled

Referential coupling means:

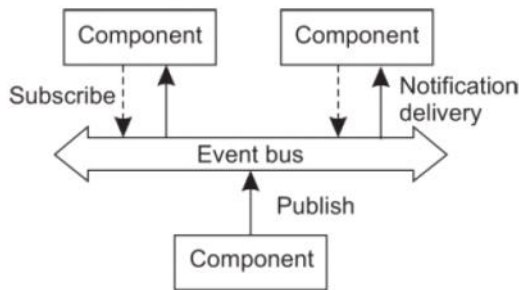
- Components know each other → referentially coupled
- Components do NOT know each other → referentially decoupled



Coordination styles	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

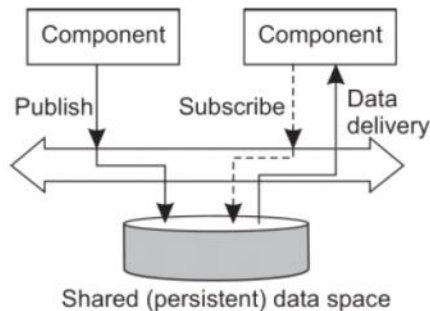
Event Bus

- Publisher sends an **event** , Subscribers receive it **immediately**
- It is **temporally coupled**



Shared Data Space

- Publisher **stores data** ,Subscribers can **retrieve it later**
- It is **temporally decoupled**



Middleware : A software platform sitting between applications and the operating system/network to enable communication between distributed components. !

Using legacy to build middleware

Problem

Legacy systems have old or limited interfaces, so modern I cannot use them easily.

Solution

- A **wrapper (adapter)** is placed in front of the legacy system.
- It provides a modern interface to applications.
- It **translates** new requests into the old system format.
- It **converts** old responses into a modern format.

Key Idea

The wrapper hides the complexity of the legacy system and makes it usable in modern distributed systems.

Organizing wrappers

When multiple applications must communicate, there are two ways to organize wrappers:

Two solutions

1- Direct (1-on-1)

Each application connects directly to every other application using **wrappers**.

The number of wrappers grows very fast (**$O(N^2)$**).

Problem: Becomes hard to manage when the system grows.

2- Through a Broker

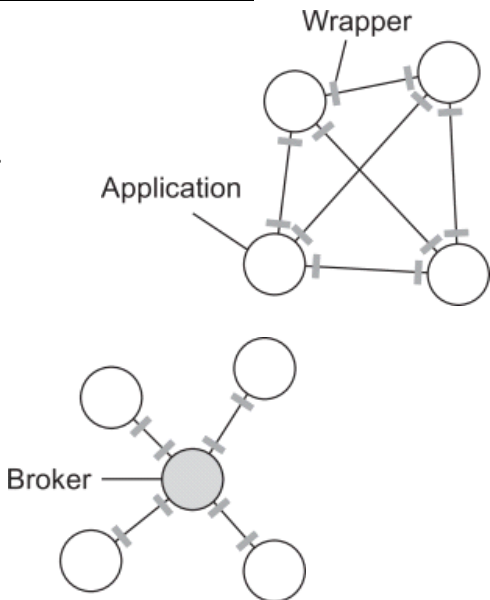
All applications communicate through a central broker.

Each application connects only to the broker.

Complexity grows linearly $\rightarrow O(N)$

Advantage: Using a broker reduces complexity and improves scalability.

Broker: A central management component that applications connect through, reducing system complexity **from $O(N^2)$ to $O(N)$** compared to direct connections. !



Developing Adaptable Middleware

Normally

A request goes from client to middleware to network to the remote object directly.

Problem

Middleware works for most applications, but sometimes we need to change its behaviour for specific cases.

Solution

- **Interceptor**: A mechanism allowing middleware to **insert extra behavior** (like logging or security) into a request without changing code. !!

What is an Interceptor?

An interceptor is placed in the middle of a request to control it before it goes to destination.

It can:

- Observe the call.
- Modify the call.
- Add extra behaviour.
- Log information.
- Check security.
- Measure performance.

Idea : It processes the request before it reaches the final destination.

Two Types

1) Request-Level Interceptor

An interceptor that acts when the request is created.

Example:

- Add authentication token
- Log who is calling
- Check permissions

2) Message-Level Interceptor

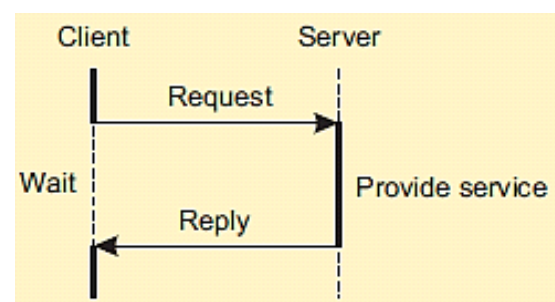
An interceptor that acts when the message is being sent.

Example:

- Encrypt message
- Compress message
- Modify headers

Centralized system architectures

Client-Server Model: An architecture where servers offer services and clients consume them using a request and reply model.



Multi-tiered Centralized System Architectures

In centralized systems, we can divide the system into layers (tiers):

- **Single-Tiered System:** An architecture where all components run on a single machine.
- **Two-Tiered System:** An architecture split into a **client** configuration and a **single server** configuration.
- **Three-Tiered System:** An architecture where the user interface, application processing, and database each reside on separate machines.

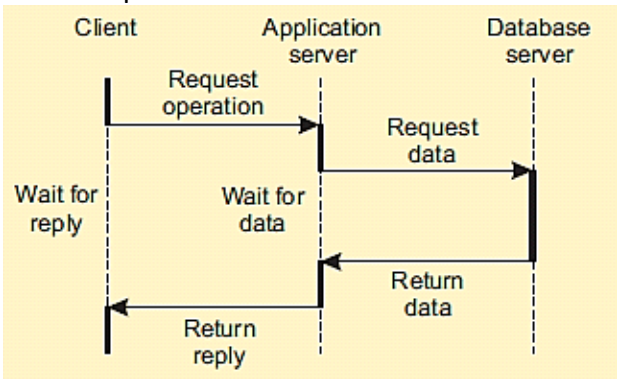
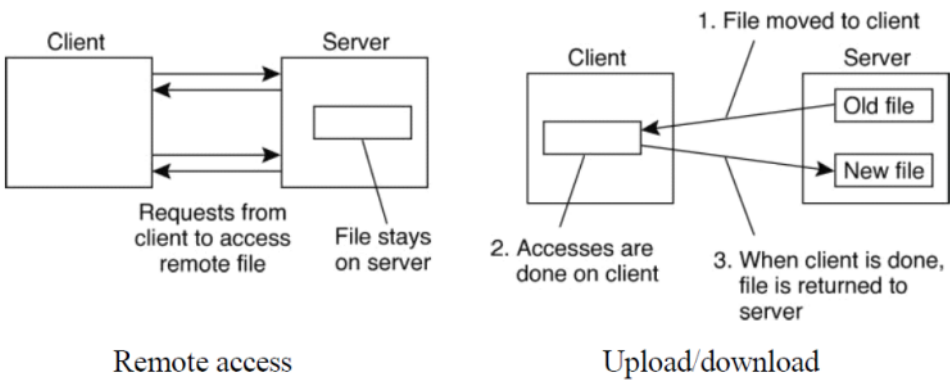


diagrama bakın

Network File System (NFS)

! NFS is a distributed file system where a client can access files on a remote server like they were on local machine.

The NFS remote access model



In distributed file systems there are two main file-access models: **remote access** and **upload/download**. NFS mainly **follows the remote access model**, and the upload/download model is shown for **comparison**.

- ! • **Remote Access Model:** A file-access model where the file remains on the server while the client interacts with it over the network. current data - good consistency
- **Upload/Download Model:** A file-access model where the entire file is moved to the client for processing and returned to the server after finishing . might be old data - sync. issue

26 Mart 2026 Perşembe 22:32

Client Side

When a program on the client tries to open a file:

1- System call layer → receives the file request (e.g., open file)

2- VFS(Virtual File System) → checks if the file is local or remote ! if it local -> directly opens the file

If remote → request goes to:

✓ NFS client - it means file is at the another machine , starts the remote access

✓ RPC client stub - sends the request to the server

→ Request is sent over the network

Server Side

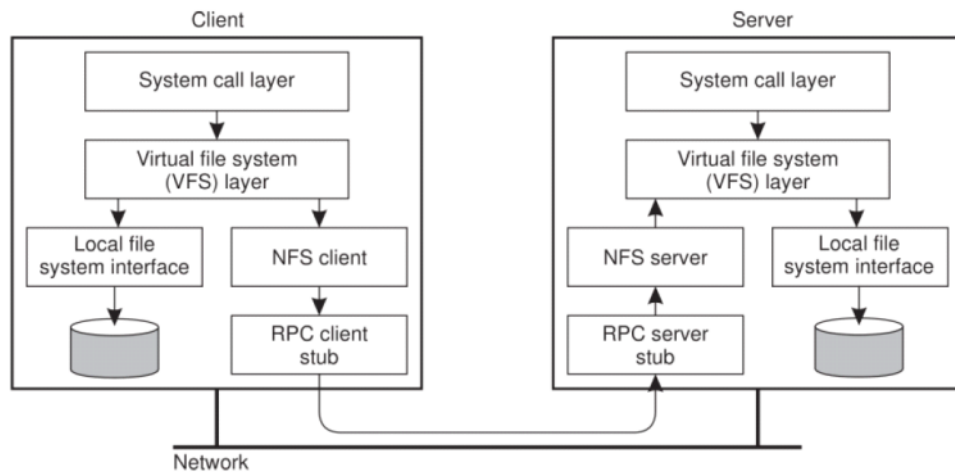
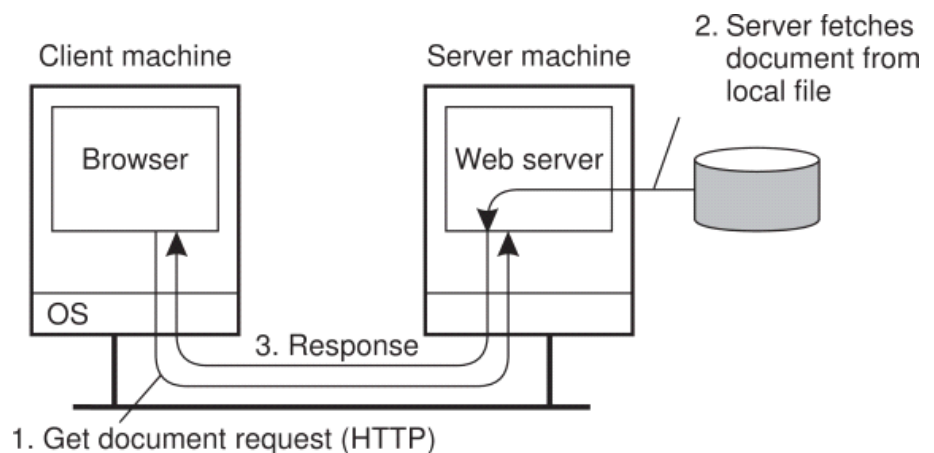
When server receives the request:

1- RPC(Remote Procedure Call) server stub → receives network request

2- NFS server → processes file request

3- Local file system → accesses actual disk

4- Response sent back to client

**Example: Simple Web servers****Back in the old days**

The client (browser) sends an HTTP request to the server.

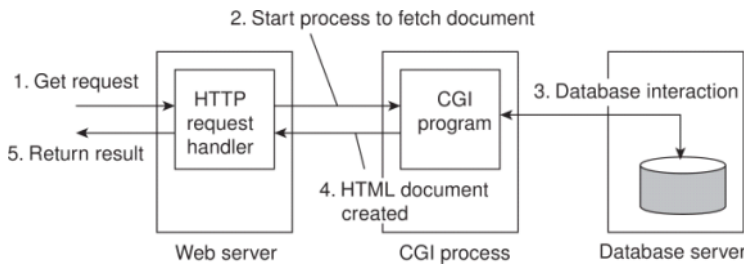
The server gets the file from its local storage.

The server sends the response back to the client.

Life was simple

- A website was just a collection of HTML files.
- HTML files were connected with hyperlinks.
- The server only needed a link to find and send a file.
- The browser was responsible for showing the content.

Example (cnt'd): Less simple Web servers



Still back in the old days

1. Client sends an HTTP request
2. Web server starts a process (CGI program)
3. CGI program communicates with database
4. HTML page is created
5. Server sends result back to client

Life became more complicated

- Website is built using a database
- Pages can still be accessed with hyperlinks
- Server uses a program (CGI) to create pages
- Content is generated dynamically (not just static files)
- Browser still displays the page

Different ways to distribute a system:

- **Vertical distribution:**
The system is divided into layers (like presentation, logic, and data). Each layer runs on a different machine.
- **Horizontal distribution:**
The system is split into similar parts. Each part works independently and has its own copy of the data.
- **Peer-to-peer (P2P):**
In peer-to-peer systems, all processes are equal.
Each process can be both a client and a server at the same time.
These processes are sometimes called “**servents**” (server + client).

Structured P2P !

Structured P2P uses a semantic-free index.

This means data is not stored by its meaning or name, but by a **key**.

- Each data item gets a unique key, usually with a **hash function**.

The system stores (**key, value**) pairs.

To find data, the request is sent to the node with that key.

A simple example is a **hypercube**.

Chord: A Structured Peer-to-Peer Overlay

A structured network where nodes are logically organized into a circular ring to allow fast data lookups.

Chord is a structured P2P system.

- Nodes are organized in a **ring (circle)**.
- Each node has an **m-bit identifier**.
- Each data item is **hashed** to an **m-bit key**.
- Data is stored at the node called the **successor** (the first node with $id \geq key$).
- The system uses **shortcut links** to find data faster.

fast data lookup

Main idea:

Chord creates an **organized circular index** for fast and reliable data search.



Unstructured P2P: A network where **nodes connect randomly** without any strict, underlying organization.

- Nodes connect **randomly**
- There is **no fixed structure**
- The network is **not organized**

How is data found ? -> **Flooding Algorithm :**

- A node sends a request to **all neighbors**
- Each neighbour forwards to request to all other neighbours ,Thus the request spreads in the network
- The process continues until node with the data sends a reply or TTL limit is reached.
- To stop infinite spreading, **TTL (Time To Live)** is used

Flooding:

- Finds data **quickly**
- Has **very high communication cost**
- Has **poor scalability**

Improvement: **Random Walk Algorithm :**

- A node sends a file request to only a **few randomly chosen neighbors**

Random Walk:

- Uses **less communication**
- **Slower to find data**
- Has **better scalability**



Super-peer Networks

Super-peer networks:

- Most nodes are **normal peers**
- Some selected nodes act as **super-peers**

Super-peers:

- They **manage** a group of normal peers
- They **keep indexes** (information about data)
- They **handle search** coordinations

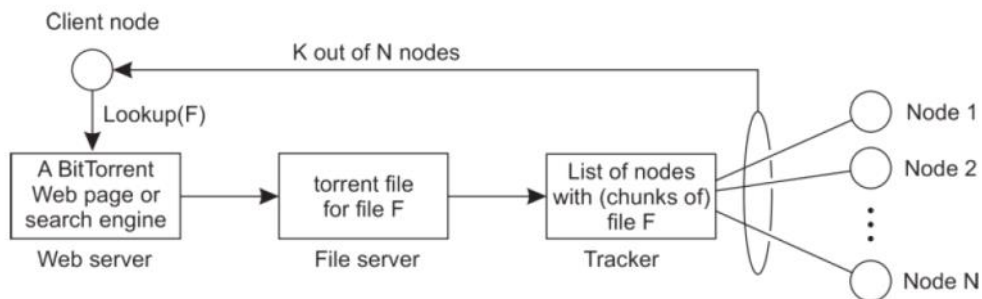
Advantages:

- More **efficient searching** than flooding
- **Reduced network traffic**
- **Better scalability**

Trade-off:

- There is some partial **centralization vulnerabilities**
- If a **super-peer fails**, its group is affected

The BitTorrent case



BitTorrent:

- BitTorrent is a **distributed file-sharing system**
- It uses:
 - A **central tracker** (for coordination)
 - **Peer-to-peer data transfer**

How it works:

- **User wants a file**
 - The user downloads a **.torrent file**
 - It contains file metadata and tracker address
- The client **contacts the tracker**
 - The tracker keeps a list of peers who have file
 - The tracker does **not store the file**
- The user **downloads from peers**
 - User : connects directly to the peers and downloads pieces of the file
 - and simultaneously uploads pieces to others

👉 Each peer is both:

- **Client** (downloads)
- **Server** (uploads)

Important idea:

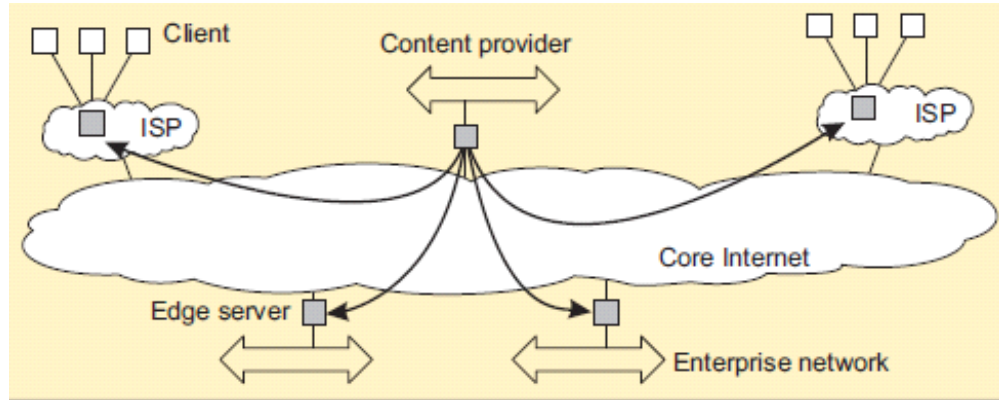
- BitTorrent is a **hybrid system**
(not fully centralized, not fully decentralized)

Why important:

- No central server stores the file
- Load is shared between peers
- It **scales better**

BAKMA GEÇ

Edge-Server Architecture



- ! In edge-server architecture, servers are placed at the edge of the network, closer to end users, instead of only in centralized data centers.

Reasons for using edge infrastructure:

1. Latency and Bandwidth:

Placing servers closer to users:

- Reduces communication delay (latency)
- Reduces network traffic
- Improves real-time apps (video calls, games)

3. Reliability:

Internet links can fail or be unstable.

If services are distributed at the edge:

- Failures localized
- System is less likely to fail
- Availability is better

2. Security and Privacy:

Edge servers can:

- Processes sensitive data locally
- Sends less data to central servers
- Protects user data

- latency ↓
- traffic ↓
- reliability ↑

! Edge-server architecture distributes servers geographically to **reduce latency, improve reliability, and enhance scalability** by bringing services closer to end users.

Edge orchestration:

- It manages resource allocation, service placement, and dynamic edge selection.

1. Resource Allocation:

- Decide where a service runs
- Check CPU and memory
- Check if the node is available

2. Service Placement:

- If a node fails, move the service
- Move to another edge node or to the cloud
- Needs **dynamic management**

3. Edge Selection:

- Choose the best edge server for each user
- Usually the **nearest server**

Example:

- User in Istanbul → nearest edge in Istanbul
 - User in Berlin → nearest edge in Berlin
- The system must select the optimal edge.

👉 **Main idea:**

Edge orchestration means **managing services and resources efficiently at the edge**